

1

Another established technique for low-bit accuracy holds a few extreme weights out of the grid (Kim et al., 2024; Dettmers et al., 2024). In single-precision PTQ the held-out weights are chosen for one precision. In a multi-precision format, however, a weight set aside changes what every precision reads. Which weights to hold out therefore becomes part of designing the base grid.

We start from the embedded-quantization literature rather than from the recent formats, because it already lists the choices involved in designing a grid that is read at several precisions. The constraints of the LLM weight path settle which of those choices are open. We make the open choices under those constraints and obtain StagQ, a multi-precision weight format with two parts. A sparse side record holds the extreme weights out of the grid. The record is filled in two stages, by a pre-grid guard before the grid is fitted and by a post-grid guard after it is fixed. The main stream is a group-wise affine 2-bit base refined by signed binary planes on a dyadic step schedule (Figure 1). The base and plane fields read at any supported precision concatenate into one ordinary integer. One affine map then reconstructs the main-stream weight by scaling that integer by a step and adding an offset. Both are derived from one pair stored per group, and no per-weight lookup is needed. Because the fields are stored in plane order, loading a lower precision fetches fewer bytes.

On Llama-3.1-8B we evaluate two configurations, StagQ-XXS and StagQ-XS, against Any-Precision LLM (Park et al., 2024b), AnyBCQ (Park et al., 2026) and MatGPTQ (Kleinegger et al., 2026). On Phi-4 and OLMo-2-7B we compare StagQ-XXS. At two bits it leads the strongest multi-precision baseline on MMLU on all three models. At three bits it leads on Llama-3.1-8B, leads on Phi-4 while spending more bits, and ties on OLMo-2-7B. Our kernel is faster than the Any-Precision LLM and AnyBCQ kernels in most shape-precision cases of a batch-one matrix-vector product on an NVIDIA A100 GPU, timed on synthetic weights.

2 BACKGROUND

2.1 MULTI-PRECISION QUANTIZATION

A multi-precision weight format is an *embedded*, or multiresolution, scalar quantizer. Reading a longer prefix refines the same scalar. Under the same rate definition, distortion measure, calibration information and decoder class, every prefix-readable code is also a feasible fixed-precision code, but not conversely. The distortions are therefore ordered, and a prefix-readable format cannot beat the best fixed-precision quantizer at matched rate.

Scalar quantization (Max, 1960; Lloyd, 1982; Gish & Pierce, 1968) and its embedded form (Equitz & Cover, 1991; Brunk & Farvardin, 1996; Sullivan, 2004; Pearlman & Said, 2008) have been studied extensively. In the rate-distortion limit, successive refinability is the case where that ordering holds with equality. Equitz & Cover (1991) show that the equality holds only for particular sources and distortion measures.

The recent LLM formats are variants of this classical object and fall into three families by mechanism. *Slice nesting* takes a lower precision from the leading digits of a deeper code. MatQuant (Nair et al., 2025) achieves this nesting through training, and MatGPTQ (Kleinegger et al., 2026) through post-training quantization. Both anchor at the highest precision, and a shallow read inherits its constants. Such a read covers a fraction of the highest-precision range, and its clipped end cell folds into a wider one. SliceMoE (Choi et al., 2025) truncates an affine code and its integer zero-point by the same bit shift. *Plane and residual nesting* adds a separately scaled term per precision. AnyBCQ (Park et al., 2026) reuses previously assigned binary codes, RRQ (Luo et al., 2026) quantizes the residual left by the preceding stages, and D2MoE (Wang et al., 2025) pairs an affine base with binary refinement. Their metadata grows with the number of precisions exposed. Single-precision BPDQ (Chen et al., 2026) fits an independent scale per bit plane. Its feasible level vectors fill a two-dimensional family, whereas one scale spans only a ray. A format that decodes every precision from one stored pair is confined to that ray. LUT-GEMM (Park et al., 2024a) already expresses uniform quantization on that ray, as binary planes with power-of-two scales, but feeds the planes to a lookup-table matrix kernel. *Learned lookup*

reconstruction admits irregular level sets. Any-Precision LLM (Park et al., 2024b) quantizes to its lowest precision first and reaches each higher precision by splitting the clusters of that first fit, storing one refitted lookup table per output channel per precision. The refitted tables require an indexed reconstruction and metadata that grows with the precisions served.

StagQ, proposed in this work, decodes every precision from one stored pair and reads its planes by prefix, reconstructing each shallower read at the midpoint of the cells it merges. It fits that pair jointly across every precision the stream serves.

2.2 DESIGNING THE GRID

The classical direct-sum construction of residual quantization (Barnes et al., 1996) reconstructs a value from a prefix of one stream, adding one term per plane to a base term:

$$\widehat{W}^{(M)} = c^{(M)} + \sum_{k=1}^M \alpha_k \varphi_k(d_k), \quad 1 \leq M \leq N. \quad (1)$$

The stream supports precisions up to N . A read at precision M reconstructs a weight W as $\widehat{W}^{(M)}$ from M of the N planes. Quantization is per group, with g weights sharing one set of stored constants. The base term $c^{(M)}$ sets the origin at the precision being read. The step α_k scales the term that plane k adds. The digit map φ_k gives the value of the digit d_k at that plane. Designing a grid means choosing all three. The weights constrain the choice through their statistics, while the serving system constrains it through the metadata it can store and the decoder it can run. Bit-plane image coders such as JPEG2000 (Pearlman & Said, 2008) use this construction (Appendix H).

Let u be the integer represented by the fields read at precision M . A group-wise affine grid at precision M has step s_M and origin lo_M , the value that code $u = 0$ takes under midpoint reconstruction. An interior code u corresponds to the half-open cell C_u of width s_M centered on $lo_M + u s_M$, whose lower edge we write a_u . For an interior cell,

$$r_u = \mathbb{E}[W \mid W \in C_u], \quad \delta_u = \frac{r_u - a_u}{s_M} \quad (2)$$

are the conditional mean of that cell and its position within the cell, with $\delta_u = \frac{1}{2}$ at the midpoint. Equation (1), with $c^{(M)}$ allowed to vary with the code read, takes the form

$$\widehat{W}^{(M)} = lo_M + (u + \epsilon_u) s_M, \quad \epsilon_u = \delta_u - \frac{1}{2}. \quad (3)$$

The same three choices reappear here in affine coordinates. The base term $c^{(M)}$ gives lo_M together with the displacement ϵ_u , the steps $\{\alpha_k\}$ give the spacing s_M , and the digit maps $\{\varphi_k\}$ give the integer u . The next section fixes all three for StagQ in this order.

3 THE STAGQ FRAMEWORK

The encoder splits the weights into two parts. A sparse side record P carries the entries that the two guards hold out of the quantizer, one before the grid is fitted and one after. The pre-grid guard records the value it removes, and the post-grid guard records the residual the fixed grid leaves. The rate charges each held-out entry for its position. The main stream, the multi-precision part, carries everything else. The complete reconstruction is

$$\widehat{W}^{(M)} = (1 - m) \odot \widehat{W}_{\text{main}}^{(M)} + P, \quad (4)$$

where m marks the held-out entries. At a pre-grid entry, P is the recorded value. At a post-grid entry it is the two-bit reconstruction plus the recorded residual. Neither depends on M , and one P therefore serves every precision. Both the grid design and the prefix property belong to the main stream, which carries a two-bit base followed by one-bit refinement planes. The evaluated instance sets $N = 6$, exposing five read precisions, $p2$ through $p6$. Each added plane raises the payload by one bit per weight. A larger N only appends further planes and adds no metadata.

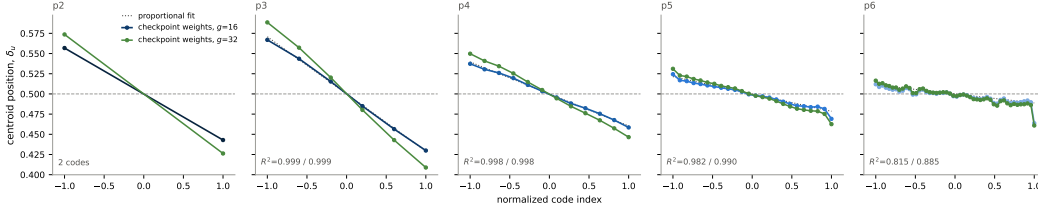


Figure 2: Measured position δ_u of the conditional centroid inside each code cell, at five precisions on a common vertical scale. Blue: Llama-3.1-8B at $g = 16$; green: $g = 32$. Further checkpoints are in Appendix D.

3.1 RECONSTRUCTION POINTS FROM MEASURED WEIGHTS

The first choice to fix is the base term $c^{(M)}$ of Equation (1), which sets the reconstruction point inside the cell selected by the fields read at precision M . For fixed boundaries and squared error, the best position is the conditional centroid r_u of Equation (2). A closed form for r_u requires a density. The $\delta \approx 0.38$ of JPEG2000 (Pearlman & Said, 2008), for example, assumes a Laplacian one. Grouped LLM weights have no established density of that kind, and we therefore measure r_u on the checkpoint weights. The measurement partitions each group with the conventional lattice of four levels, the outer two on the group minimum and maximum (Appendix D).

Figure 2 plots the measured δ_u . The dashed line marks the midpoint, where a flat within-cell density puts the centroid. A uniform control recovers the midpoint at every precision and at both group sizes StagQ uses, $g = 16$ and $g = 32$. Because only two cells remain at $p2$, the shape is read from $p3$ on. Through $p5$, the centroids fall on a straight line that crosses $\frac{1}{2}$ at the group center. Each centroid is displaced toward the center in proportion to its distance from the center. A displacement of that form needs no per-code correction, since at any one precision a rescaling of the grid absorbs it.

The pair (lo, s) of each group is quantized with two sixteen-entry codebooks fitted per matrix, one in $\log_2 s$ and one in lo/s . Together they span 256 combinations, and each group then takes the combination that minimizes the loss across every precision the stream serves. For weights W_c reconstructed at precision M as $\widehat{W}_c^{(M)}$, and with every precision weighted equally,

$$D(lo, s) = \frac{1}{N-1} \sum_{M=2}^N \sum_c E_c (W_c - \widehat{W}_c^{(M)})^2 + \lambda \Phi(lo, s), \quad (5)$$

where E_c is the calibration mean square of input channel c (column c of W), and Φ is a floor term,

$$\Phi(lo, s) = \sum_c E_c \max(|W_c - \widehat{W}_c^{(2)}| - \rho, 0)^2, \quad (6)$$

in which $\rho = s \sum_{k=3}^N 2^{-(k-1)}$ is the furthest the refinement planes can move the base reconstruction. We set $\lambda = 4$. The sum runs over the whole group, with any entry the pre-grid guard holds out clipped to the next value inward. The floor term Φ charges only the part of the base error that no plane can reach. Because the coarse two-bit read dominates the first term, that term alone would underweight an error whose damage shows at the fine reads.

Across the seven projections of four sampled layers of Llama-3.1-8B, the fitted grids clip a few extreme weights in exchange for finer levels across the body of each group. In these fitted grids, no level sits on a group extreme.

3.2 STEP SCHEDULES UNDER A METADATA BUDGET

The second choice to fix is the refinement step α_k of Equation (1), for each plane $k \geq 3$. To avoid fitting and storing one scale per refinement stage, StagQ fits only the base step s and derives every refinement step from it.

Suppose a refinement bit splits an interval cell into two interval children, and the density inside the parent cell is uniform at the scale of the split. The minimum-squared-error design is then the midpoint split. Applied at every plane, this gives the dyadic schedule of refinement half-steps

$$t_k = \alpha_k = \frac{s}{2^{k-1}}, \quad k = 3, \dots, N. \quad (7)$$

At precision M the cell spacing is $s_M = s/2^{M-2}$, twice the additive half-step t_M . Appendix C states when this schedule is optimal. On Llama-3.1-8B, a global rescale γ of its steps lowers the median output error by at most 0.6%. The released format keeps γ at 1, which leaves the schedule of Equation (7) unchanged.

3.3 DIGIT MAPS AND LAYOUT FOR THE KERNEL

The third choice is the digit maps φ_k , which translate stored bits into numbers, and we fix them together with the physical layout. Both follow from the decode interface of an LLM, which requires fixed-width addressability, affine decoding with no per-weight lookup, and the prefix property. Fixed-width addressability restricts each field to a fixed alphabet $\mathcal{A}_k = \{0, \dots, 2^{m_k} - 1\}$, with m_k a format constant. A field's location is then an affine function of the weight index, and its unpack width is known when the kernel is built.

Affine decoding with no per-weight lookup restricts each digit map to an affine function. Our instance takes a two-bit base, $q \in \{0, 1, 2, 3\}$, with the identity as its digit map, followed by one-bit refinement digits $b_k \in \{0, 1\}$, each encoding the sign of the current residual. Therefore, with the half-steps t_k of Equation (7), each digit map is $\varphi_k(b) = 2b - 1$, and plane k then adds $2t_k b_k - t_k$. The constants these planes accumulate depend only on the precision read and the base step, and thus none of them is stored. A zero residual takes $\text{sign}(0) := +1$, and the reconstruction still moves by one half-step. Because the fitted grids of Section 3.1 keep the levels off the group extremes, a ternary alphabet is never needed. The two bits of q followed by $b_3, \dots, b_M$ form the composite integer

$$u_M = q 2^{M-2} + \sum_{k=3}^M b_k 2^{M-k}, \quad (8)$$

which is the abstract u of Equation (3). Main-stream reconstruction is therefore a scale-and-offset of one integer.

To keep the prefix property in the bytes, StagQ stores the base first and then the refinement planes in order of k . A shallower load therefore skips the deeper planes. Our kernel instead reads contiguous M -bit digits, repacked once at load for one precision (Appendix G).

3.4 THE RESULTING STAGQ MAIN STREAM

With all three choices fixed, substituting the two-bit base, the signed one-bit refinements, and the unscaled dyadic schedule into the additive reconstruction of Equation (1) gives

$$\widehat{W}_{\text{main}}^{(M)} = lo + sq + \sum_{k=3}^M (2b_k - 1) \frac{s}{2^{k-1}}, \quad M \in \{2, \dots, N\}, \quad (9)$$

with the sum empty at $M = 2$. All data-dependent terms are integer multiples of the half-step $t_M = s/2^{M-1}$ of the deepest plane read, and they combine into the composite integer u_M of Equation (8). With $s_M = s/2^{M-2}$ and $lo_M = lo - s(2^{M-2} - 1)/2^{M-1}$, the reconstruction collapses to

$$\widehat{W}_{\text{main}}^{(M)} = lo_M + s_M u_M. \quad (10)$$

Every precision therefore decodes as one affine map of one ordinary integer, with (lo_M, s_M) derived in closed form from the single stored pair (lo, s) . This map covers the main stream only. The side record P of Equation (4) is applied outside this map, and a complete runtime must fuse P or report its traffic separately.

3.5 HOLDING OUT THE EXTREMES

How to treat the extreme weights must be settled before the pair of Equation (5) is fitted, because one embedded grid serves every precision. Consider a group with one weight far larger than the rest. Widening the pair (l, s) to cover that weight spreads the four base levels apart, so every precision decodes the group more coarsely. Instead, fitting the pair to the rest leaves the extreme weight past the outermost of those four levels, where the base clips it. Because the planes reach at most ρ past that level (Equation (6)), no precision can recover a weight lying further out. For example, on Phi-4’s 80 attention matrices at $g=32$, the entries StagQ-XXS selects remove 26.9% of the two-bit error when held out before the grid is fitted, but only the 6.2% they carry when held out after it (Appendix E).

The pre-grid guard takes at most one entry per group, under one of two criteria that the configuration assigns to each projection. The ratio criterion holds out a group’s largest-magnitude entry when it exceeds the second largest by more than a factor of T , four on Llama-3.1-8B. The gain criterion weighs the two-bit error an entry’s removal saves against the storage the entry occupies. The post-grid guard runs after the grid is fixed and takes a second set, scored on the error that grid leaves behind.

To score a candidate at position j , the gain criterion clips it to the next value inward and refits the matrix codebooks and pairs. The gain Δ_j is the fall this causes in the group’s two-bit error D_2 , and the candidate is held out when its gain per bit of storage exceeds a threshold τ :

$$\Delta_j = D_2(V) - D_2(V^{j \rightarrow}), \quad \Delta_j / b > \tau. \quad (11)$$

Here V holds the group’s values, $V^{j \rightarrow}$ the same values with the candidate clipped, and b the bits the entry occupies. One τ per stream serves both guard stages. Because Δ_j includes what the refit recovers and depends on the other groups’ candidates, it is a heuristic score.

3.6 THE ENCODER

The encoder first rescales each input channel with an AWQ-style activation-aware scale (Lin et al., 2024). A calibration pass measures the mean square of each channel’s activations, and channels with larger activations are scaled up so that the grid resolves their weights more finely. The quantizer then operates on the scaled matrix. Columns are partitioned into groups of g , and each group’s pair is fitted as Section 3.1 describes. The two guard stages of Section 3.5 hold weights out of the quantizer, following standard practice (Kim et al., 2024; Dettmers et al., 2024). Each refinement plane $k = 3, \dots, 6$ stores the sign of the residual left by the preceding prefix and skips every position the side record already carries. No deeper prefix can therefore overwrite a held-out entry. Figure 3 maps every stage to the field it fills.

Every read carries the whole side record. On Llama-3.1-8B, for example, StagQ-XS spends 0.13 bits per weight on the entries its gain criterion holds out, the same at every read. A deeper read cannot drop them, because the pair was fitted with the extremes clipped, and the main stream there encodes only the clipped value. A quantizer built for six bits alone would extend its grid over these weights and store no record at all. This overhead is the cost of serving every precision from one checkpoint.

4 EXPERIMENTAL RESULTS

4.1 EXPERIMENTAL SETTINGS

We report two encoder configurations, StagQ-XS and the lower-rate StagQ-XXS. At two bits StagQ-XS runs at 2.51 to 2.64 bits per weight (bpw) and StagQ-XXS at 2.34 to 2.35 bpw, within 0.04 bpw of the strongest two-bit baseline on all three models. Each refinement plane adds exactly 1 bpw. Appendix B gives the group sizes, codebook-fitting constants, scaling exponents and the pre-grid guard settings of both configurations.

We evaluate on three dense models, Llama-3.1-8B (Grattafiori et al., 2024), Phi-4 (Abdin et al., 2024) and OLMo-2-7B (OLMo Team et al., 2025). Both configurations are reported

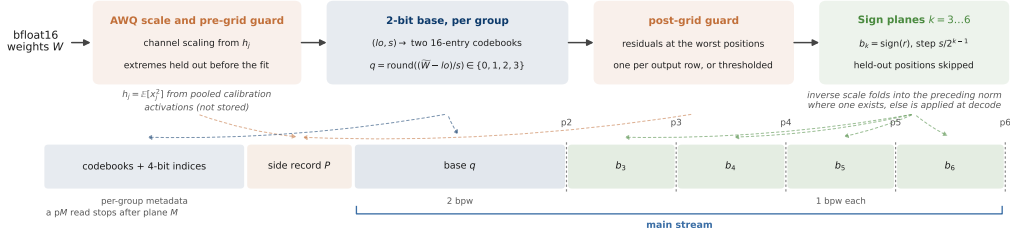


Figure 3: The StagQ encoder and the fields its rate charges. Dashed lines in each stage’s color map the stage to the field it fills. A pM read stops after plane M.

on Llama-3.1-8B. Every linear layer of the transformer blocks is quantized. The embedding and the language-model head stay in bfloat16, as in the baselines’ releases. We report 5-shot MMLU (Hendrycks et al., 2021) and WikiText-2 (Merity et al., 2017) perplexity, and on Llama-3.1-8B also five zero-shot commonsense-reasoning (CSR) tasks with their average (the CSR average). We ran each entry ourselves under one harness, in bfloat16. Every perplexity we report is token-normalized over non-overlapping 2,048-token blocks, as in the baselines’ releases. These token-level perplexities are therefore not comparable to a word-level perplexity. The baselines are MatGPTQ, Any-Precision LLM and AnyBCQ, each read from one multi-precision checkpoint by a prefix cut. Rate is the logical bpw of the payload, the side record and every side-information field a format declares, computed from the declared widths. The StagQ rate also charges two mechanisms no baseline here uses, the finer groups and the two guard stages, whose settings are chosen per model and configuration, with the hold-out thresholds also set per block. On OLMo-2-7B we chose the thresholds by rate alone. On Llama-3.1-8B and Phi-4 we also compared candidate settings on WikiText-2 perplexity and MMLU, and on Llama-3.1-8B also on the CSR average. At two bits on Llama-3.1-8B our kernel, loaded with real weights, reads 2.47 bpw against the 2.35 charged. Most of the gap comes from sizing the per-row guard slots of each feed-forward matrix to its fullest row (Appendix A).

4.2 ACCURACY EVALUATION

At two and three bits, StagQ-XXS scores higher on MMLU than the strongest baseline at that precision on all three models (Tables 1 and 2). However, its 0.9-point lead on OLMo-2-7B at three bits falls inside our one-point tie band on MMLU. StagQ-XXS is also cheaper than that baseline in five of these six model-precision cases. The exception is Phi-4 at three bits, where it costs about a third of a bit more than Any-Precision LLM and leads it by 1.2 MMLU points, an upper bound on the equal-rate advantage. On the Llama-3.1-8B CSR average at two and three bits, StagQ-XXS leads the strongest baseline by 1.3 and 1.8 points, outside our tie band of 0.6 points on that average. On Llama-3.1-8B at two bits, StagQ-XS costs a quarter of a bit more than AnyBCQ and leads it by 13.3 MMLU points, again an upper bound on the equal-rate advantage. These comparisons are between complete encoder configurations. They do not separate the format from the finer groups and the guards, which no baseline here uses. The groups of 32 stay cheap because each group stores one eight-bit pair that every precision reads. This pair costs 0.25 bpw, less than the 0.375 to 0.625 bpw that AnyBCQ’s per-precision records cost at two to four bits with groups of 128.

From four bits up the strongest baseline is Any-Precision LLM, which is 0.15 to 0.30 bpw cheaper than StagQ-XXS. At four bits StagQ-XXS leads it by under one MMLU point. At five and six bits on Llama-3.1-8B it trails by up to 0.4 points on MMLU and on the CSR average. We count all of these as ties. StagQ-XXS has lower perplexity on the C4 (Raffel et al., 2020) validation shard than every baseline at two to four bits on Llama-3.1-8B and OLMo-2-7B. In two of these cases the best baseline on C4 is cheaper. At five and six bits on Llama-3.1-8B, Any-Precision LLM has a lower rate and a lower C4 perplexity.

Table 1: Accuracy on Llama-3.1-8B, in AnyBCQ’s column order (HS is HellaSwag and WG WinoGrande). Bit is the read precision M and Rate the logical bpw. Bold marks the best quantized entry per column within a precision.

Method	Bit	Rate	MMLU	ARC-C	ARC-E	HS	PIQA	WG	CSR Avg
bfloat16	16	16.00	65.23	53.67	81.06	78.91	81.28	73.72	73.73
MatGPTQ	2	2.25	24.65	24.74	24.45	26.29	51.41	49.17	35.21
Any-Precision	2	2.01	23.82	24.57	34.09	27.90	57.24	50.36	38.83
AnyBCQ	2	2.38	35.33	37.29	62.67	62.76	73.94	57.93	58.92
StagQ-XXS	2	2.35	38.38	38.05	60.94	63.63	73.23	65.27	60.23
StagQ-XS	2	2.64	48.67	41.98	70.12	70.05	75.52	67.32	65.00
MatGPTQ	3	3.25	44.84	42.41	67.97	68.39	74.05	67.32	64.03
Any-Precision	3	3.03	55.69	47.27	74.58	73.36	78.67	67.72	68.32
AnyBCQ	3	3.50	58.13	46.93	75.88	74.37	79.16	69.46	69.16
StagQ-XXS	3	3.35	61.89	49.32	76.60	76.57	79.65	72.45	70.92
StagQ-XS	3	3.64	62.55	51.02	77.48	77.71	80.41	71.43	71.61
MatGPTQ	4	4.25	59.29	48.98	76.43	75.57	78.94	71.11	70.21
Any-Precision	4	4.05	63.55	52.73	80.26	78.21	80.79	72.06	72.81
AnyBCQ	4	4.63	63.17	51.71	78.91	77.28	80.47	73.09	72.29
StagQ-XXS	4	4.35	64.11	52.56	79.84	78.63	80.47	73.24	72.95
StagQ-XS	4	4.64	64.44	52.30	80.18	79.11	80.85	73.88	73.26
Any-Precision	5	5.10	64.94	53.07	81.14	78.74	81.18	73.01	73.43
StagQ-XXS	5	5.35	64.73	52.56	80.56	78.94	81.45	72.93	73.29
StagQ-XS	5	5.64	65.07	52.99	80.98	79.33	80.74	73.40	73.49
Any-Precision	6	6.20	65.20	53.41	81.19	79.03	81.18	73.56	73.67
StagQ-XXS	6	6.35	64.83	52.99	80.68	78.95	81.07	73.40	73.42
StagQ-XS	6	6.64	65.07	53.75	81.23	79.40	81.01	73.88	73.85

The guards matter most at low bits (Table 3). On a Llama-3.1-8B stream held at $g=32$ throughout, both guards together lower the two-bit perplexity from 28.47 to 20.86, still behind AnyBCQ’s 19.03 but at a lower rate. Two-bit MMLU is near chance in every row. The guards add 2.3 and 1.6 MMLU points at three and four bits, and only 0.35 and 0.03 at five and six. Every read at those precisions still carries their side record.

Table 4 measures the accuracy cost of the prefix constraint only within our own encoder, on Llama-3.1-8B. We refit StagQ-XXS for one precision at a time and hold every other setting fixed. At two bits the dedicated fit spends slightly more rate and scores 3.6 MMLU points higher, an upper bound on the equal-rate cost. At three and four bits the dedicated fit is cheaper by under 0.01 bpw. The MMLU differences there, -0.03 and $+0.72$ points, bound the cost from below. Both lie inside our tie band.

Table 4: Single-precision control on Llama-3.1-8B. Δ MMLU is the dedicated fit minus StagQ-XXS.

Bit	Wiki $\downarrow$		Δ MMLU
	StagQ-XXS	dedicated	
2	12.77	11.99	+3.62
3	7.24	7.18	-0.03
4	6.57	6.44	+0.72

4.3 KERNEL EVALUATION

Table 5 reports the median speedup of the three quantized kernels over cuBLAS in a batch-one matrix–vector product on the A100, across the nine shapes of AnyBCQ’s kernel table. Each baseline runs at the group size of its released kernel, and StagQ at $g=32$. The StagQ kernel is faster than the Any-Precision LLM kernel in 24 of 27 shape–precision cases and than the AnyBCQ kernel in 23. All seven comparisons it loses fall on the three attention shapes, the only ones timed with per-group guard space. Omitting that space on the six feed-forward shapes favors StagQ. These timings use synthetic weights. On a real two-bit Llama-3.1-8B stream, our decode path raises perplexity on the first four WikiText-2 blocks by 0.0086 against the same weights in **fp16**, well inside a 0.10 tolerance.

Table 2: MatGPTQ (MatQ), Any-Precision LLM (AP), AnyBCQ (AB) and StagQ-XXS (XXS). Tok/s is the mean of six timings in one run. OLMo-2-7B timings omit every per-channel multiply, favoring us.

Bit	Wiki ↓					MMLU ↑					tok/s ↑			
	bf16	MatQ	AP	AB	XXS	bf16	MatQ	AP	AB	XXS	fp16	AP	AB	XXS
<i>Llama-3.1-8B</i>														
2	6.24	>10 ⁶	4674.38	19.03	12.77	65.23	24.65	23.82	35.33	38.38	46.16	73.91	74.15	73.68
3	6.24	11.07	8.58	8.09	7.24	65.23	44.84	55.69	58.13	61.89	46.16	70.10	70.16	70.88
4	6.24	7.48	6.70	6.84	6.57	65.23	59.29	63.55	63.17	64.11	46.16	66.32	66.36	68.31
<i>Phi-4</i>														
2	6.46	9.4×10 ⁵	17.71	9.69	8.84	80.34	24.65	46.38	66.66	70.68	29.00	52.98	54.40	53.86
3	6.46	7.23	7.17	7.63	6.80	80.34	75.76	77.26	76.76	78.41	29.00	49.74	50.65	50.55
4	6.46	6.71	6.63	7.32	6.54	80.34	79.29	79.50	79.01	79.92	29.00	46.22	47.20	48.81
<i>OLMo-2-7B</i>														
2	6.10	4.8×10 ⁵	5.5×10 ⁴	11.74	8.98	62.78	22.95	25.51	46.03	53.01	50.50	82.12	83.88	85.50
3	6.10	8.11	7.35	7.28	6.68	62.78	57.68	57.53	59.15	60.08	50.50	78.16	79.46	80.49
4	6.10	6.66	6.38	6.48	6.24	62.78	61.47	61.91	61.44	62.68	50.50	73.64	75.13	78.44

Table 3: Guard ablation, Llama-3.1-8B, $g=32$. The side-record column is the rate the guards add. Every row stores the payload bits and 0.25 bpw of per-group pairs.

Bit	side record	Wiki ↓					MMLU ↑				
	(bpw)	2	3	4	5	6	2	3	4	5	6
neither guard	0	28.47	7.60	6.66	6.43	6.38	26.90	58.03	62.62	64.29	64.74
post-grid only	0.0056	23.18	7.47	6.58	6.36	6.31	28.51	58.63	63.11	64.47	64.88
pre-grid only	0.0010	24.77	7.57	6.69	6.48	6.43	25.00	59.74	63.84	64.40	64.78
both	0.0065	20.86	7.44	6.60	6.41	6.36	25.09	60.30	64.26	64.64	64.77

In Table 2, we evaluate end-to-end decode speed. Our path fuses format-independent operators that the baseline paths run separately, as in their releases. With these fusions and all decoders timed in one run, our path is faster in fourteen of the eighteen comparisons. Timed at three bits without the fusions or the per-channel multiply, it is slower than the fastest baseline on all three models, by 0.3, 0.6 and 2.9%. All three quantized decoders are far faster than the **fp16** decoder, ours by a factor of 1.48 to 1.86.

Table 5: Median speedup over cuBLAS, t_{cuBLAS}/t_* , on synthetic weights. The StagQ kernel here applies no activation scale. Its layout reserves guard space only on the three attention shapes. Appendix F gives the per-shape latencies.

Bit	Any-Precision	AnyBCQ	StagQ
2	3.53×	3.93×	4.13×
3	2.76×	3.01×	3.39×
4	2.34×	2.43×	3.00×

5 DISCUSSION AND CONCLUSION

StagQ serves every precision from one prefix-readable main stream, decoded by one affine map with no per-weight lookup. Two guards hold out the few weights the grid serves worst in a sparse side record. On Llama-3.1-8B, Phi-4 and OLMo-2-7B, StagQ-XXS leads the strongest multi-precision baseline on MMLU at two bits, at a slightly lower logical rate. At three bits it leads on Llama-3.1-8B, leads on Phi-4 at a higher rate, and ties on OLMo-2-7B. From four bits up it ties Any-Precision LLM on MMLU at a higher rate. On synthetic weights, our matrix-vector kernel is faster than both baseline kernels in most shape-precision cases. With format-independent fusions and on synthetic weights, our decode path is faster in fourteen of eighteen end-to-end comparisons. Unfused at three bits, it trails the fastest baseline.

Future work will extend the evaluation to larger models and to mixture-of-experts models, and will ablate the parts of the encoder one at a time. On the systems side, with further engineering, the kernel can be taken into production serving, where one deployment switches among precisions at run time.

ACKNOWLEDGMENTS

We used generative AI tools to help develop the conceptual framework and to help write the proof of Appendix C. We also used them to write the code, to interpret results, to draft and edit the text and to produce the figures. We reviewed all AI-assisted work, checked every number from our own runs against the underlying result files, verified every derivation independently of the tools, and take responsibility for the final content of this paper.

REFERENCES

- Marah Abdin et al. Phi-4 technical report. *arXiv preprint arXiv:2412.08905*, 2024.
- Christopher F. Barnes, Syed A. Rizvi, and Nasser M. Nasrabadi. Advances in residual vector quantization: A review. *IEEE Transactions on Image Processing*, 5(2):226–262, 1996.
- Hugh Brunk and Nariman Farvardin. Fixed-rate successively refinable scalar quantizers. In *Proceedings of the Data Compression Conference (DCC)*, pp. 250–259, 1996.
- Junyu Chen, Jungang Li, Jing Xiong, Wenjie Wang, Qingyao Yang, He Xiao, Zhen Li, Taiqiang Wu, Mengzhao Chen, Zhen Peng, Chaofan Tao, Long Shi, Hongxia Yang, and Ngai Wong. BPDQ: Bit-plane decomposition quantization on a variable grid for large language models. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. arXiv:2602.04163.
- Yuseon Choi, Sangjin Kim, Jungjun Oh, Gwangtae Park, Byeongcheol Kim, and Hoi-Jun Yoo. SliceMoE: Bit-sliced expert caching under miss-rate constraints for efficient MoE inference. *arXiv preprint arXiv:2512.12990*, 2025. Accepted to the Design Automation Conference (DAC) 2026.
- Tim Dettmers, Ruslan Svirschevski, Vage Egiazarian, Denis Kuznedelev, Elias Frantar, Saleh Ashkboos, Alexander Borzunov, Torsten Hoefer, and Dan Alistarh. SpQR: A sparse-quantized representation for near-lossless LLM weight compression. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2306.03078.
- William H. R. Equitz and Thomas M. Cover. Successive refinement of information. *IEEE Transactions on Information Theory*, 37(2):269–275, 1991.
- Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. OPTQ: Accurate quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR)*, 2023. Also known as GPTQ; arXiv:2210.17323.
- Herbert Gish and John N. Pierce. Asymptotically efficient quantizing. *IEEE Transactions on Information Theory*, 14(5):676–683, 1968.
- Aaron Grattafiori et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations (ICLR)*, 2021.
- Sehoon Kim, Coleman Hooper, Amir Gholami, Zhen Dong, Xiuyu Li, Sheng Shen, Michael W. Mahoney, and Kurt Keutzer. SqueezeLLM: Dense-and-sparse quantization. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pp. 23901–23923, 2024. arXiv:2306.07629.
- Maximilian Kleinegger, Elvir Crnčević, and Dan Alistarh. MatGPTQ: Accurate and efficient post-training matryoshka quantization. *arXiv preprint arXiv:2602.03537*, 2026.
- Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024.

- Stuart P. Lloyd. Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982.
- Yu Luo, Bo Dong, Wenhua Cheng, and Haihao Shen. Recurrent residual quantization: A progressive multi-precision representation for LLMs. *arXiv preprint arXiv:2608.04048*, 2026.
- Joel Max. Quantizing for minimum distortion. *IRE Transactions on Information Theory*, 6(1):7–12, 1960.
- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *International Conference on Learning Representations (ICLR)*, 2017.
- Pranav Ajit Nair, Puranjay Datta, Jeff Dean, Prateek Jain, and Aditya Kusupati. Matryoshka quantization. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pp. 45484–45506, 2025.
- OLMo Team et al. 2 OLMo 2 furious. *arXiv preprint arXiv:2501.00656*, 2025. Shorter version in Conference on Language Modeling (COLM), 2025.
- Gunho Park, Baeseong Park, Minsub Kim, Sungjae Lee, Jeonghoon Kim, Beomseok Kwon, Se Jung Kwon, Byeongwook Kim, Youngjoo Lee, and Dongsoo Lee. LUT-GEMM: Quantized matrix multiplication based on LUTs for efficient inference in large-scale generative language models. In *International Conference on Learning Representations (ICLR)*, 2024a.
- Gunho Park, Jeongin Bae, Beomseok Kwon, Byeongwook Kim, Se Jung Kwon, and Dongsoo Lee. AnyBCQ: Hardware efficient flexible binary-coded quantization for multi-precision LLMs. In *International Conference on Learning Representations (ICLR)*, 2026.
- Yeonhong Park, Jake Hyun, SangLyul Cho, Bonggeun Sim, and Jae W. Lee. Any-precision LLM: Low-cost deployment of multiple, different-sized LLMs. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pp. 39682–39701, 2024b.
- William A. Pearlman and Amir Said. Image wavelet coding systems: Part II of set partition coding and image wavelet coding systems. *Foundations and Trends in Signal Processing*, 2(3):181–246, 2008.
- Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140): 1–67, 2020.
- Gary J. Sullivan. On embedded scalar quantization. In *Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, volume 4, pp. 605–608, 2004.
- Haodong Wang, Qihua Zhou, Zicong Hong, and Song Guo. D2MoE: Dual routing and dynamic scheduling for efficient on-device MoE-based LLM serving. In *Proceedings of the 31st Annual International Conference on Mobile Computing and Networking (ACM MobiCom)*, 2025.

A LIMITATIONS

Rate. Every rate we report counts the bits that the format’s fields declare. On Llama-3.1-8B at two bits the kernel loaded with real weights reads 2.47 bits per weight against the 2.35 charged. Our loader stores the guard entries of the feed-forward matrices in per-row slots, and every row of a matrix carries as many slots as its fullest row. A side record indexed

by output row would read 2.39 bits per weight. One that also marks each attention group with a presence bit would read 2.354, within 0.004 of the charge. Both need a kernel that decodes records of variable length. We leave that engineering change to future work.

Scope of the evidence. All quality results come from three dense models, and none from a mixture-of-experts model. Most of the encoder analysis rests on the 28 matrices of four sampled layers of Llama-3.1-8B. The GEMV and decode timings are batch-one on one A100 with synthetic weights. Real weights cost our kernel slightly more shared-memory conflict than synthetic weights do. At two bits on Llama-3.1-8B, decoding with real weights reads about a fifth of the `fp16` weight bytes. By bytes alone, decoding could therefore run 4.7 times as fast as `fp16`. However, the decoder we time reads fewer bytes still and decodes only 1.6 times as fast.

Comparisons. Because no configuration lands on a baseline’s rate, every comparison with a baseline is made across a rate difference. Several baseline runs also depart from their authors’ releases. We supplied the model configurations for Any-Precision LLM on Phi-4 and OLMo-2-7B and for AnyBCQ on OLMo-2-7B. Outside its released recipe, we ran MatGPTQ asymmetrically and added two bits to the precisions it optimizes jointly. We score AnyBCQ’s released Phi-4 checkpoint, and our own AnyBCQ runs on OLMo-2-7B, with each model’s own tokenizer rather than with the tokenizer files the release’s code saves beside the checkpoint. Scored with those saved files instead, our AnyBCQ MMLU on Phi-4 comes within 0.05 points of the value Park et al. (2026) report, against up to 0.32 points with Phi-4’s own tokenizer. Our AnyBCQ run on OLMo-2-7B uses the per-device training batch size of two from the release’s README, and a run at batch size one differs from it by at most 0.17 MMLU points. Our Any-Precision LLM runs also disagree with the numbers Park et al. (2026) report for it at two bits. We obtain WikiText-2 perplexity 4674 against their 1681 on Llama-3.1-8B, and 17.71 against their 13.37 on Phi-4. At three and four bits our Any-Precision LLM perplexities agree with theirs within 0.03.

Ablations and controls. We set the group sizes and guard criteria per model and configuration, and the hold-out thresholds per model, configuration and block (Appendix B). We report no ablation of λ , of the AWQ exponents or of how the pair (l, s) is fitted. Apart from the single-precision control, every fit we report weights the precisions of Equation (5) equally. Among the streams reported here, the only one with nothing held out is the neither-guard row of Table 3, on Llama-3.1-8B at $g=32$. In the same table, adding the pre-grid guard on top of the post-grid guard raises WikiText-2 perplexity at four to six bits by 0.02 to 0.05. Outside the table, the same addition raises C4 perplexity at three to six bits by 0.09 to 0.11. Because the both-guards row also spends more rate than the post-grid-only row, these increases would be at least as large at equal rate. The single-precision control covers Llama-3.1-8B only and measures the cost of the prefix constraint within our own encoder. It therefore does not bound that cost against the best fixed-precision quantizer.

B ENCODER CONFIGURATIONS

Table 6 lists the encoder settings of the two reported configurations. Group sizes and guard criteria are set per model and configuration. Hold-out thresholds are also set per block (attention or feed-forward). Phi-4 fuses $q/k/v$ into one projection and gate/up into another, and its attention block therefore carries one group size.

The remaining settings are shared by all models and configurations. The gain criterion always scores the activation-weighted two-bit error. The AWQ scale a_c of input channel c is proportional to $\mathbb{E}[x_c^2]^{\alpha/2}$, with $\alpha = 0.25$, raised to 0.5 on the down projections. The codebooks of each matrix are fitted to an initial pair per group, which is never stored. The initial pair places the four levels symmetrically, with a step of 0.29 of the group range and the outer levels 6% in from either end. On the layers sampled in Section 3.1, the fitted grids also stay narrower than the group range.

Table 6: Encoder settings per model and configuration. Each pre-grid guard column names the matrices under that criterion and its threshold. A dash marks a criterion that no matrix uses. The two criteria are exclusive per matrix. On Phi-4 the fused gate/up projection carries only the post-grid guard.

Model	Configuration	Feed-forward g	Attention g	Pre-grid guard	
				ratio criterion	gain criterion
Llama-3.1-8B	StagQ-XS	16	16	feed-forward, $T=4$	attention, $\tau=10^{-8}$
	StagQ-XXS	32	v 16, q, k, o 32	feed-forward, $T=4$	attention, $\tau=10^{-9}$
Phi-4	StagQ-XS	16	16	attention, down, $T=4$	—
	StagQ-XXS	32	32 (fused)	down, $T=4$	attention, $\tau=10^{-8}$
OLMo-2-7B	StagQ-XS	16	16	all, $T=8$	—
	StagQ-XXS	32	32	feed-forward, $T=4$	attention, $\tau=10^{-7}$

C OPTIMALITY OF THE DYADIC SCHEDULE

We show that the dyadic schedule of Equation (7) is optimal when (i) every code cell is an interval, (ii) one refinement bit splits it into two interval children, (iii) the conditional density inside the parent cell is uniform, and (iv) each child is reconstructed by one scalar under squared error.

Proposition. Let X be uniform on $[a, b]$, split at a threshold θ into $[a, \theta]$ and $[\theta, b]$, each reconstructed by one value. The minimum mean-squared-error design is the midpoint split,

$$\theta^* = \frac{a+b}{2}, \quad r_L^* = \frac{3a+b}{4}, \quad r_R^* = \frac{a+3b}{4}. \quad (12)$$

Proof. For fixed θ the best reconstructions are the child centroids, $r_L = (a + \theta)/2$ and $r_R = (\theta + b)/2$. The distortion is then

$$D(\theta) = \frac{(\theta - a)^3 + (b - \theta)^3}{12(b - a)}, \quad (13)$$

whose derivative vanishes only at $\theta = (a + b)/2$. Because $D'' = \frac{1}{2} > 0$, the midpoint is the unique minimizer.

Recursion. A uniform density restricted to either child is again uniform. The proposition therefore applies at every plane. Starting from a base cell of width s , plane k moves the reconstruction by $\pm t_k$ with $t_k = s/2^{k-1}$. At precision M the cells have width $s_M = s/2^{M-2}$. The recursion thus recovers the schedule of Equation (7), which sets the half-step of Equation (9) and the spacing of Equation (10).

Scope. The recursion covers only the refinement planes above the four-level base, which is fitted in Section 3.1. Under a nonuniform conditional density, the optimal threshold and reconstructions move off the midpoints.

Testing the schedule against the displacement. The centroid displacement of Section 3.1 shows that condition (iii) fails. We test whether it calls for a different schedule with one format-wide factor γ on the refinement steps, $\alpha_k = \gamma s/2^{k-1}$, which adds no per-group metadata. The cost is

$$C(\gamma) = \frac{1}{4} \sum_{M=3}^6 \frac{D_M(\gamma)}{D_M(1)}, \quad D_M = \mathbb{E}_x \|(W - \widehat{W}^{(M)})x\|^2, \quad (14)$$

where D_M is the output error of the precision- M reconstruction under calibration activations x drawn from the same pool as the base fit. In this test the pre-grid guard uses the ratio criterion ($T=4$) on every matrix. The base is held at its $\gamma = 1$ fit under the first term of Equation (5) alone, without the floor term Φ that StagQ adds. We take the median of $C(\gamma)$

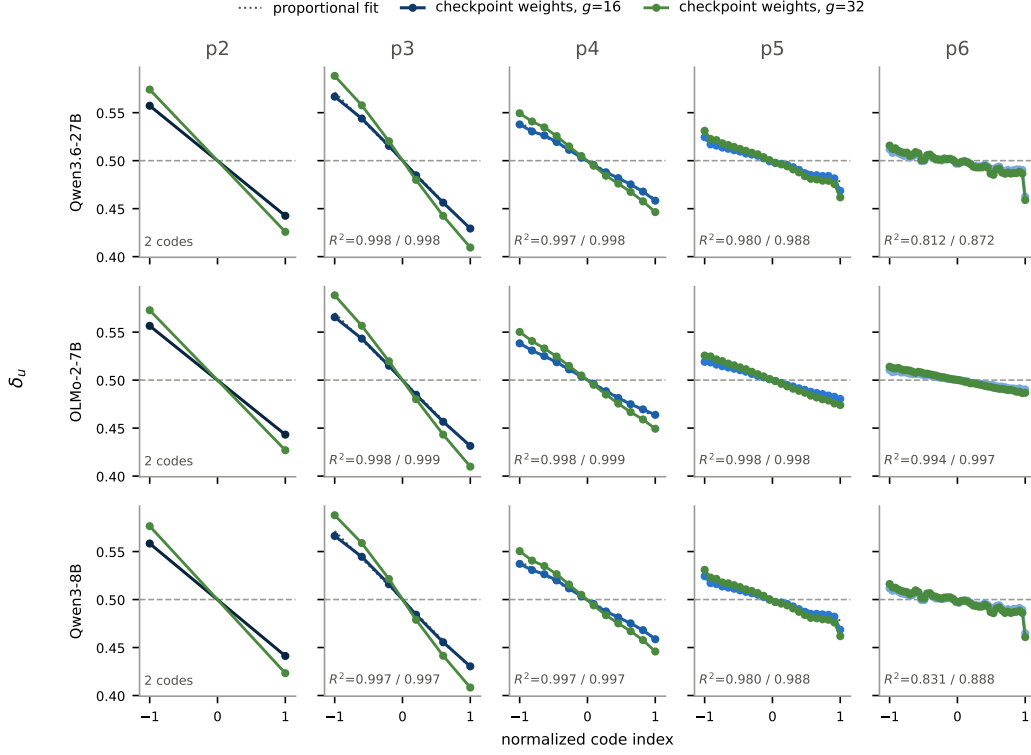


Figure 4: The measurement of Figure 2 on the three further dense checkpoints (rows: Qwen3.6-27B, OLMo-2-7B, Qwen3-8B) at five precisions (columns), on a common vertical scale. Blue: $g = 16$; green: $g = 32$. R^2 of the proportional fit (blue / green) is marked from $p3$ on.

over 28 matrices of Llama-3.1-8B at the two group sizes this paper evaluates. At $g = 16$ the median is lowest at $\gamma = 1$. At $g = 32$, however, a rescale to 1.05 lowers the median by 0.6%. The two group sizes disagree on where the optimum lies. The $g = 32$ basin is also flat, with the median at $\gamma = 1.1$ within 0.16% of that at 1.05. The released format keeps $\gamma = 1$.

D THE CENTROID SHAPE ON FURTHER CHECKPOINTS

We repeat the measurement of Section 3.1 on three further dense checkpoints, Qwen3.6-27B, OLMo-2-7B (OLMo Team et al., 2025) and Qwen3-8B (Qwen Team, 2025), at five precisions and both evaluated group sizes (Figure 4). Together with the Llama-3.1-8B checkpoint of Section 3.1, the statistics below cover four checkpoints. On each checkpoint, including that of Figure 2, we sample 24 feed-forward matrices and subsample each to evenly spaced rows, up to four million entries. We keep only the cells that lie wholly inside the group range, and drop each group’s smallest and largest entry. As a control, we replace every sampled matrix with uniform random values of the same shape. On this uniform control, δ_u lies between 0.4992 and 0.5007 for every code over the five precisions and both group sizes.

On the three further checkpoints, the straight line of Section 3.1 is reproduced at every group size from $p3$ through $p5$ (Figure 4). The proportional fit is a straight line fitted to δ_u against the code index. Across the four checkpoints, its slope at $p4$ spans -0.73 to -0.70 at $g = 16$ and -0.97 to -0.94 at $g = 32$, in units of $10^{-2} \delta_u$ per code step. From $p3$ through $p5$, R^2 lies between 0.980 and 0.999, and the crossing at $\frac{1}{2}$ sits at the group center. At these precisions the property Section 3.1 uses, a displacement proportional to the distance from the group center, therefore holds on each of the four checkpoints. However, at $p6$ the measured curve shifts below $\frac{1}{2}$ on three of the four checkpoints, and its crossing moves up

Table 7: Llama-3.1-8B at two bits, with rates in logical bpw. The $+v$ row changes only the value projection’s group size. The rows marked $\rightarrow$ change several settings at once. The $g=32$ row is the both-guards row of Table 3. AnyBCQ scores 19.03 and 35.33 at 2.38 bpw.

	bpw	Wiki $\downarrow$	MMLU $\uparrow$
$g=32$ throughout	2.26	20.86	25.09
+ v at $g=16$	2.26	16.68	29.46
$\rightarrow$ StagQ-XXS	2.35	12.77	38.38
$g=16$ throughout	2.51	10.82	43.71
$\rightarrow$ StagQ-XS	2.64	9.68	48.67

to 0.16 of the half-width off the center. On these three, R^2 drops to 0.81–0.83 at $g = 16$ and 0.87–0.89 at $g = 32$. OLMo-2-7B is the exception, with $R^2 = 0.994$ at $g = 16$ and 0.997 at $g = 32$.

E ENCODER STAGES

Scale. The encoder calibrates all three reported models on one pool of 200 fixed 2,048-token C4 (Raffel et al., 2020) windows. The quantizer operates on the AWQ-scaled matrix (Lin et al., 2024) $W \text{diag}(a)$, with $a_c \propto \mathbb{E}[x_c^2]^{\alpha/2}$ for input channel c and AWQ exponent α . The inverse scale folds into a preceding normalization where one exists. Where none exists, the decoder we time applies this inverse scale at run time on Llama-3.1-8B and Phi-4 and omits it on OLMo-2-7B.

Base. Each of the two **fp16** codebooks of Section 3.1 is placed by one of three rules. The rules are uniform placement over the range of the initial pairs (Appendix B), Lloyd’s rule, and Lloyd’s rule with its ends anchored at that range. For each matrix, the encoder picks one of the nine rule combinations by the first term of Equation (5) alone. It adds the floor term only in the later per-group choice among the 256 pairs. However, a matrix whose six-bit error exceeds a tenth of its two-bit error under the chosen codebooks falls back to uniform placement. The base code is nearest rounding, with no GPTQ error propagation or feedback update.

Guards. The pre-grid guard is that of Section 3.5, under the criteria of Table 6. On a matrix under the ratio criterion, and on Phi-4’s fused gate/up projection, the post-grid guard takes the one position per output row with the largest activation-weighted two-bit error. Under the gain criterion, it scores each position by that error per stored bit, takes the three highest-scoring positions per row and keeps those whose score exceeds τ . The 26.9% and 6.2% of Section 3.5 are measured on Phi-4’s 80 attention matrices at $g=32$, under the gain criterion at $\tau=10^{-8}$.

Side-record storage. Under the gain criterion, a pre-grid entry is an eight-bit symmetric fixed-point code against one **fp16** scale per output row. Its position is charged at $\log_2(n/k) + \log_2 e$ bits for a matrix of n weights holding out k . This charge is slightly above the per-entry length of an enumerative code of the positions. However, the encoder does not store such a code. The layout our kernel reads keeps fixed per-group slots on attention and per-row slots on the feed-forward matrices instead (Appendix A). Under the ratio criterion, a pre-grid entry is **fp16** with a flat index. A post-grid entry is its **fp16** residual against the two-bit base, with a $\lceil \log_2 c \rceil$ -bit index of its column among the c inputs. Under the gain criterion, each output row also carries a two-bit count of its post-grid entries.

Group sizes. Table 7 compares StagQ-XXS and StagQ-XS at two bits with the $g=32$ -throughout and $g=16$ -throughout configurations they start from. At three bits, they gain 1.59 and 0.61 MMLU points over those configurations, which score 60.30 (Table 3) and 61.94. The second gain lies inside the tie band. From four bits up, no MMLU or CSR difference from the starting configuration exceeds 0.42 points in magnitude.

Table 8: Batch-one GEMV latency (μs) on one A100 (40 GB) for the nine shapes (input $\times$ output) of AnyBCQ’s kernel table, from Llama-3-8B, Phi-4 and Llama-3-70B in row order. Bold marks the fastest quantized kernel per shape and width.

	cuBLAS	Any-Precision				AnyBCQ			StagQ		
Shape	16-bit	2-bit	3-bit	4-bit	2-bit	3-bit	4-bit	2-bit	3-bit	4-bit	
8B qkv/o 4096×4096	29.4	11.5	14.3	16.6	12.2	15.3	17.9	13.2	14.7	14.6	
8B down 14336×4096	97.8	29.2	38.6	47.1	25.1	32.5	40.2	24.1	29.0	32.6	
8B gate/up 4096×14336	97.9	27.3	34.7	41.9	24.9	32.5	40.7	23.7	28.9	32.8	
14B qkv/o 5120×5120	50.1	15.5	18.7	22.5	14.3	18.2	22.2	16.3	18.4	20.0	
14B down 17920×5120	141.3	40.8	48.4	54.4	32.1	37.7	49.4	30.8	36.4	41.6	
14B gate/up 5120×17920	137.5	33.7	43.6	48.7	30.2	40.1	47.8	28.6	37.9	41.5	
70B qkv/o 8192×8192	104.9	34.8	39.3	47.8	26.6	35.2	43.7	28.9	34.3	37.8	
70B down 28672×8192	344.1	80.9	124.8	120.0	62.0	86.2	110.5	58.8	78.7	94.4	
70B gate/up 8192×28672	340.7	68.3	91.2	114.4	62.9	85.3	109.5	59.1	78.8	94.5	

F PER-SHAPE GEMV LATENCY

The cuBLAS column of Table 8 is the median of three timings per shape, one at each bit width. Table 5 divides each kernel’s timing by the cuBLAS timing taken at the same width. Its speedups can therefore differ slightly from those recomputed from this column. On the three attention shapes, the StagQ layout we time reserves an extra half bit per weight for per-group guard space. It reserves none on the six feed-forward shapes, although real feed-forward matrices also hold guard entries. The feed-forward timings therefore favor StagQ over the other kernels. On the attention shapes, the StagQ kernel loses five of the six comparisons at two bits. It wins ten of the twelve at three and four bits.

G KERNEL IMPLEMENTATION

Our kernel is one CUDA C++ GEMV that uses no CUTLASS, cuBLAS or tensor cores. It keeps the tiling, half-precision accumulation and atomic-add structure of the AnyBCQ kernel, so that a timing difference comes from the format and not from the surrounding code. A thread block of 256 threads owns a 1024-column output tile and a 64-row input tile. In our decode path, each block loads into shared memory a 256-entry table for each codebook pair its output rows use. The table maps each combination of the two codebooks of Section 3.1 to (l, s) . The lookup runs once per group of g weights. The kernel timed in Table 5 reads this table from constant memory instead. The kernel does not read the stored **fp16** tensors. A separate converter builds integer bit planes from the base levels, codebook indices and codebooks, which the encoder can also write. The kernel reads these planes as contiguous M -bit digits, repacked once at load. With real weights we ran this path at two bits only. Both guard stages are applied in the same pass without branching.

H THE GRID OF A BIT-PLANE IMAGE CODER

JPEG2000 quantizes a wavelet coefficient y with step Δ by the dead-zone quantizer

$$q_J = \text{sign}(y) \lfloor |y|/\Delta \rfloor. \quad (15)$$

The magnitude $|q_J|$ is written in N_b bits and coded one bit-plane at a time, most significant first, with b_k the bit at plane k . Reading M planes reconstructs a coefficient whose read bits are not all zero (an all-zero read reconstructs to zero) as

$$\hat{y}^{(M)} = \text{sign}(y) \Delta \left(\sum_{k=1}^M b_k 2^{N_b-k} + \delta 2^{N_b-M} \right), \quad (16)$$

which is Equation (1) under $\alpha_k = \text{sign}(y) \Delta 2^{N_b-k}$, $\varphi_k(b_k) = b_k$, and $c^{(M)} = \text{sign}(y) \delta \Delta 2^{N_b-M}$.

The steps α_k and digit maps φ_k only write the integer $|q_J|$ in binary. The base term $c^{(M)}$ places the reconstruction inside the cell the unread planes leave open, at a fraction δ of its width. The fraction δ is therefore the only design choice in this grid. Under the Laplacian model commonly used for wavelet detail coefficients, the reconstruction that minimizes squared error sits at a fixed fraction of every cell read at one precision. Pearlman & Said (2008) report that a single $\delta \approx 0.38$, measured on the magnitude from the cell edge nearer zero, works well in practice.